

STRINGS I

"There has never been a
meaningful life built on
easy street."

~ John Paul Warren



Good

Evening

On saturday → PSP session

Today's content → Introduction

→ Flip

→ check if string is palindrome

→ Longest Palindromic substring

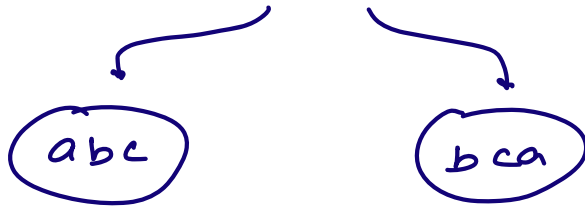
Explain it
on saturday { → Reverse a string word by word

→ count sort on characters

String → Array of characters

→ Series of characters

~~X~~ → Group of character



Characters → Decimal value assigned to every character {ASCII Table}

'A' → 65

'B' → 66

'C' → 67

'D' → 68

⋮

'z' → 90

'a' → 97

'b' → 98

'c' → 99

⋮

'z' → 122

'0' → 48

'1' → 49

'2' → 50

⋮

'9' → 57

'10' → Combination of char

Quiz → `System.out.print('a'+1);`

97 + 1

98

Output = 98

`(char)(98) → 'b'`

String str = 'a' + 2 + " ";
 $\downarrow$
 $97 + 2 + " "$
 $99 + " " \rightarrow "99"$

String str = 'a' + " " + 2 :
 $"a" + 2 \rightarrow "a2"$

Addition of
 anything to
 a string will
 result into
 concatenation

String str = 2 + 3 + " "
 $5 + " " \rightarrow "5"$

String str = " " + 2 + 3
 $"2" + 3 \rightarrow "23"$

String str = 'a' + 'b' + " ";
 $= 97 + 98 + " ";$
 $= 195 + " " \rightarrow "195"$

01. Given a string, toggle every character of string

↓
uppercase to lowercase

lowercase to uppercase

Note :- String only contains lowercase or uppercase characters.

Eg:- sCaLeR → SCALeR

Eg:- aBcdE → AbCdE

Ideas → Check if a character is in lowercase or uppercase

↓

Convert it to lowercase

↓

Convert it to uppercase

(a) Check if a character is in uppercase

// $ch \geq 65 \ \&\& \ ch \leq 90$

// $ch \geq 'A' \ \&\& \ ch \leq 'Z'$

Conver to lowercase (char) $[ch + 32]$

(b) Check if a character is in lowercase

// $ch \geq 97$ && $ch \leq 122$

// $ch \geq 'a'$ && $ch \leq 'z'$

Conver to Uppercase `(char)(ch - 32)`

String Flip (String str)

String ans = "";

for (i=0; i<n; i++)

char ch = str.charAt(i)

if ($ch \geq 'A'$ && $ch \leq 'z'$)

ans = ans + (char)(ch + 32);

else {

ans = ans + (char)(ch - 32);

return ans;

}

Constraint $\rightarrow$ You can't use if & else condition

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0		2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
A = 65	0	1	0	0	0	0	0	1		a(97)	0	1	1	0	0	0	1
B = 66	0	1	0	0	0	0	1	0		b(98)	0	1	1	0	0	1	0
C = 67	0	1	0	0	0	0	1	1		c(99)	0	1	1	0	0	1	1
D = 68	0	1	0	0	0	1	0	0		d(100)	0	1	1	0	0	1	0

$+32$
 -32

uppercase $\xrightarrow{5^{\text{th}} \text{ bit set}}$ lowercase
 lowercase $\xrightarrow{5^{\text{th}} \text{ bit unset}}$ uppercase

String Flip (String str)

String ans = "";

for (i=0; i<n; i++)

char ch = str.charAt(i)

char at ith idx

// ans = ans + (char)(ch ^ (2⁵));

ans = ans + (char)(ch ^ (1<<5));

3

return ans;

1 = 000000.0001

1<<1 = 00000001

1<<2 = 00000010

1<<3 = 00000100

$$\begin{array}{r}
 \Delta = \begin{array}{cccccccc}
 2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 \\
 \hline
 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\
 \hline
 \end{array} \\
 1 \ll 5 = \begin{array}{cccccccc}
 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
 \hline
 \end{array} \\
 a \rightarrow \begin{array}{cccccccc}
 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1
 \end{array}
 \end{array}$$

— ^ — ^ — ^ —

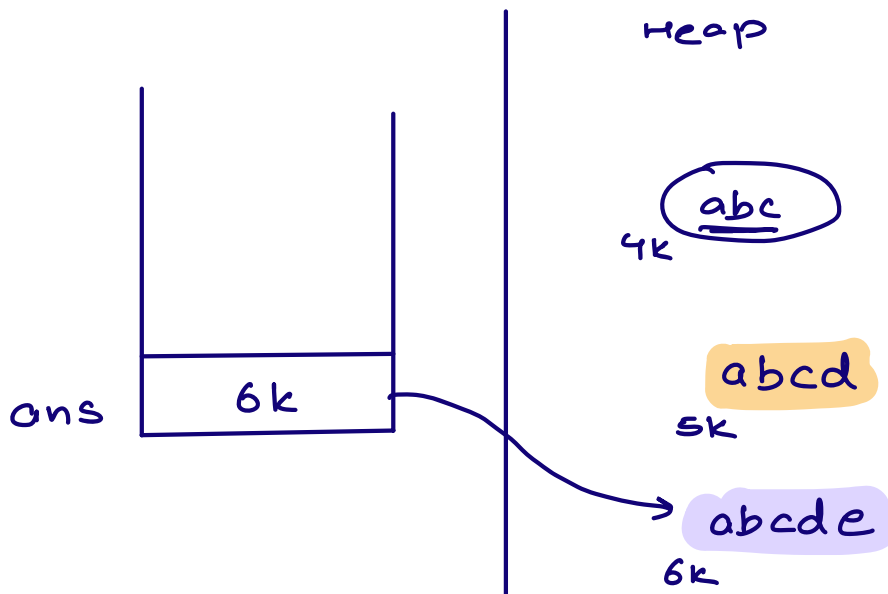
Immutable → Strings in Java are immutable

Once defined, can't change its value

→ String ans = "abc";

ans = ans + 'd';

ans = ans + 'e';



StringBuilder → Mutable String

→ `StringBuilder sb = new StringBuilder();`

01. Access the character → `sb.charAt(idx);`

02. Add to a stringbuilder → `sb.append('a');`

03. Add at particular idx → `sb.insert(idx, 'y');`

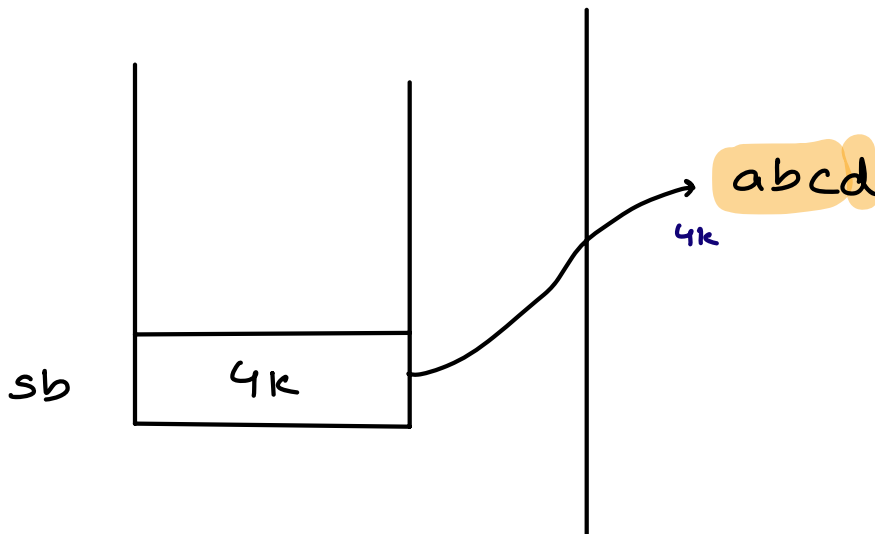
04. Change at particular idx → `sb.setCharAt(idx, 'd');`

05. Delete at particular idx → `sb.deleteCharAt(idx);`

06. `StringBuilder` to `String` → `sb.toString();`

`sb = "abc"`

`sb = sb.append(d);`



Substring → Continuous part of a string

Full string is also a substring

single character is also a substring

02. Check if a given substring is **palindrome** or not?

01. madam
02. woow
03. civic } Palindrome [2 S]

String str = "a n a m a d a m s p e"
0 1 2 3 4 5 6 7 8 9 10

P₁ P₂
↓ ↓

substring = [3, 7]

Idea → Use two pointers to check if a substring is palindrome or not.

```
boolean isPali (String str, int st, int e)
{
    while (s < e)
    {
        if (str.charAt(s) != str.charAt(e)) return false;

        s = s + 1;
        e = e - 1;
    }
    return true;
}
```

Tc: O(n)
Sc: O(1)

Q Given a string, calculate length of longest palindromic substring

Constraint :- $1 \leq N \leq 3 * 10^3$

Eg:- abacabd
0 1 2 3 4 5 6

Ans = 5

str = abcd
0 1 2 3

Ans = 1

String str = "anamadamape" Ans = 7
0 1 2 3 4 5 6 7 8 9 10

Brute force → Generate all the substrings & check if the substring is palindrome & update the max length:

No. of substring * TC to check if a substring is palindrome

$$= \frac{n(n+1)}{2} * n$$

$$= n^2 * n$$

$$TC = O(n^3)$$

$$SC = O(1)$$

Code → {TODO}

Optimised Idea

Str:-

a	b	c	c	b	d
0	1	2	3	4	5
↑					↑
P ₁					P ₂

c
↓

$$\text{len} = 6 - 0 - 1 = 5$$

$[p_1, p_2]$ - both included

$$\text{len} = p_2 - p_1 + 1$$

(P_1, P_2) - both excluded

$$\text{len} = P_2 - P_1 + 1 - 2$$

$$\text{len} = p_2 - p_1 - 1$$

str = c a b b a d

 1 2 3 4 5

 ↑ ↑

 P₁ P₂

$$\text{len} = 5 - 0 - 1 = 4$$

str = { x b d y z z y d b d y z y d x }

Idea 2 - Take every character as a center & try to extend as much as you can & update your max

Even length
palindrome { Take every adjacent pair as a center & try
to extend as much as you can & update
your max

```
int longpal (string str)
```

```
int ans = 0
```

```
for (i=0; i<n; i++) { // odd length palindromes
```

```
    // center = i
```

```
    P1 = i    P2 = i
```

```
    int len = expand (str, P1, P2)
```

```
    ans = Math.max (ans, len);
```

```
}
```

```
for (i=0; i<n-1; i++) { // even length palindromes
```

```
    // center = i & center = i+1
```

```
    P1 = i    P2 = i+1
```

```
    int len = expand (str, P1, P2)
```

```
    ans = Math.max (ans, len);
```

```
}
```

```
}
```

$$TC = O(n^2 + n^2) = O(n^2)$$

$$SC = O(1)$$

```
int expand (str, p1, p2)
```

```
while ( p1 ≥ 0 && p2 < n && str.charAt(p1) == str.charAt(p2))
```

```
    p1 = p1 - 1;
```

```
    p2 = p2 + 1;
```

```
return p2 - p1 - 1;
```

String str = a b a a b

0 1 2 3 4

↑ ↑
p₁ p₂

↓

ans = 0 1 3 4

Reverse the String

You are given a string A of size N.

Return the string after reversing the string word by word.

Note :- Reversed string should not contain leading or trailing spaces

→ If there are multiple spaces between words, reduce them to a single space in reversed string.

Eg:- "the - sky - - is - blue - - -"

Eg:- "- this - - is - me -"

blue - is - sky - the

me - is - this

$$a + b = 3$$

Doubts session

Arr = { 5 6 7 8 9 }

↑
a

$$\text{diff} = 3$$

$$a - b = 3$$

↑

Ⓐ

$$b = a - 3$$

Arr = { 1 1 1 1 4 }

$$\text{diff} = 3$$

1 → 4

4 → 1

Ⓐ

$$Am = \left\{ \begin{array}{cc} \overset{\frown}{1} & \overset{\frown}{2} \\ \cup & \cup \\ \underset{\smile}{1} & \underset{\smile}{2} \end{array} \right\} \quad B=0$$

$$Ans = 3$$